DLL Developer Hub Prototype - Version 1 Review

HCI 584 - Andrea Bierstedt - Fall 2026

1. Project overview

The DLL Developer Hub Prototype gives developers one starting place to learn about DLL's finance APIs and to request (simulated) access to them. It has two main areas:

- API Catalog - browse or search API products (Quote, Finance Application, Contract), read an overview, and view simplified API specifications (method, path, required fields, sample request/response).
- Request Credentials - submit a credential request, have it validated, receive a request ID, and later check its status and a masked sample credential.

Version 1 is a command-line app (`main.py`) that uses local JSON files in `data/`. Version 2 will be a Streamlit web app.

2. What is implemented so far

All four task vignettes from the original spec work in the command-line version:

Vignette	Status
1) Start from the home page	Done - numbered home menu with API Catalog / Request Credentials / Exit
2) Browse the catalog and view an API	Done - search by name/overview/keywords, product overview, choose an operation, view its spec
3) Request credentials	Done - pick API, operations, environment; enter details; validation; saved with request ID as "Submitted"
4) Return and check the request	Done - look up by request ID, simulated review moves it to "Credentials Available" or "Needs Information", masked key

Data files:

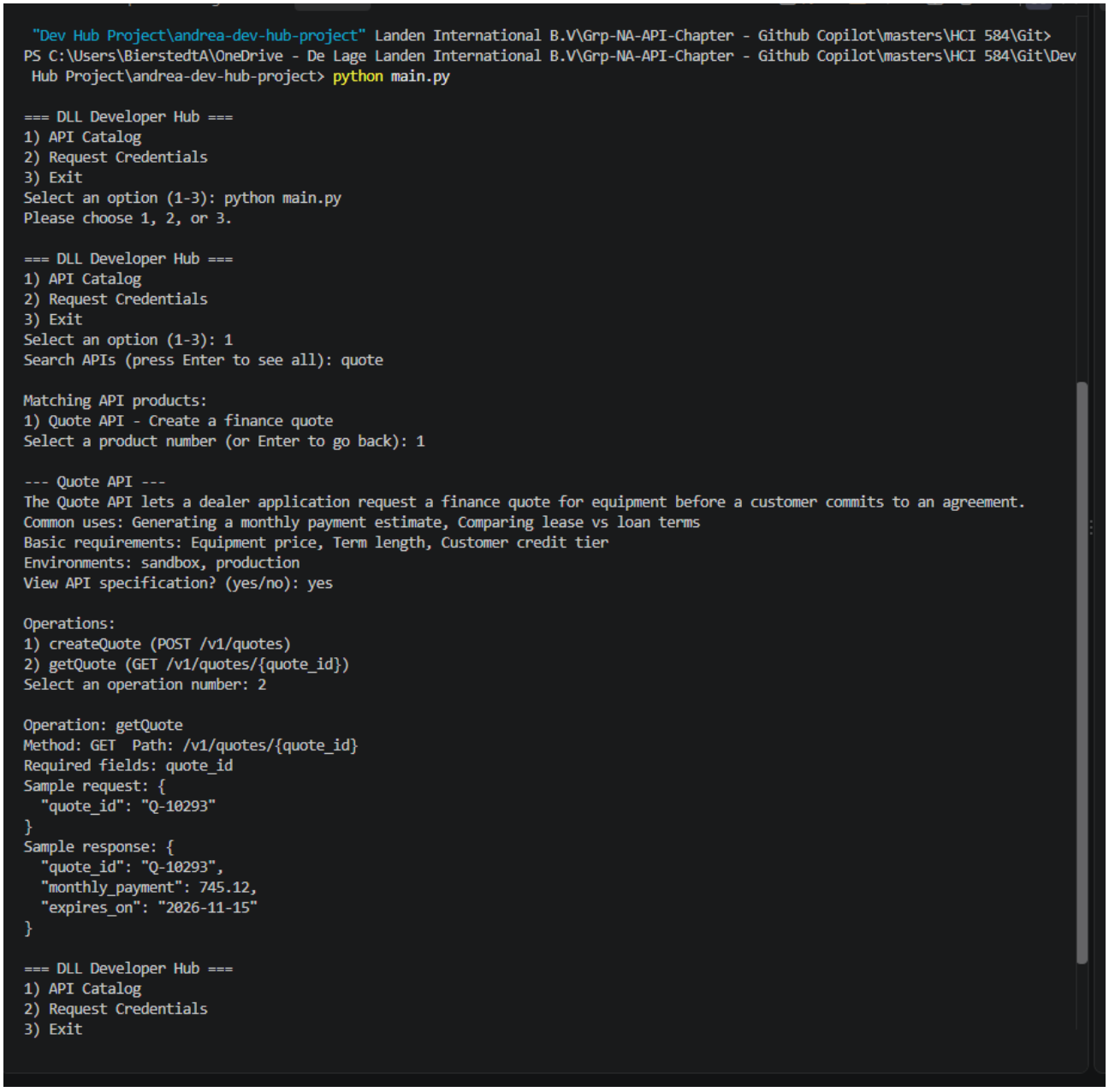
- `data/catalog.json` - 3 API products
- `data/api_specs.json` - 2 operations per product (6 total)
- `data/requests.json` - saved requests (written by the app)

3. Demo: how the app currently works

The screenshots below are from real runs of `python main.py`. Text in blue is what the user typed.

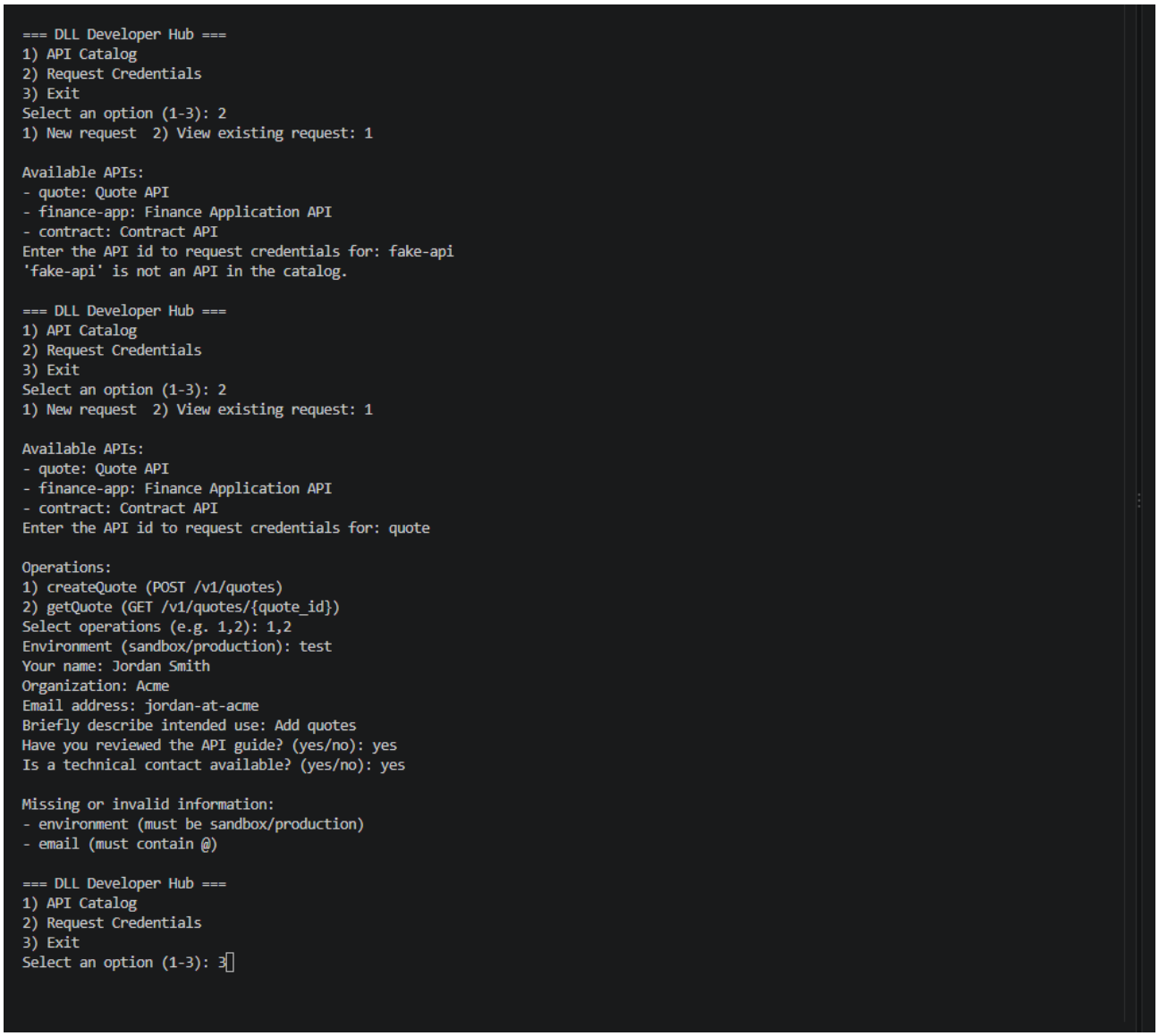
3.1 Searching the catalog and viewing an API spec

The user searches for "quote", opens the Quote API, and views the spec for the second operation (getQuote).



3.2 Validation

An API id that isn't in the catalog is rejected. Then a request with an invalid environment and email is stopped with a list of what needs fixing.



3.3 Submitting a request

A complete request for the Quote API (two operations) is saved with status Submitted and a request ID.

```
=== DLL Developer Hub ===
1) API Catalog
2) Request Credentials
3) Exit
Select an option (1-3): 3python main.py
Please choose 1, 2, or 3.

=== DLL Developer Hub ===
1) API Catalog
2) Request Credentials
3) Exit
Select an option (1-3): 2
1) New request  2) View existing request: 1

Available APIs:
- quote: Quote API
- finance-app: Finance Application API
- contract: Contract API
Enter the API id to request credentials for: quote

Operations:
1) createQuote (POST /v1/quotes)
2) getQuote (GET /v1/quotes/{quote_id})
Select operations (e.g. 1,2): 1,2
Environment (sandbox/production): sandbox
Your name: Jordan Smith
Organization: Acme Equipment
Email address: jordan@acme.com
Briefly describe intended use: Add quotes to dealer app
Have you reviewed the API guide? (yes/no): yes
Is a technical contact available? (yes/no): yes

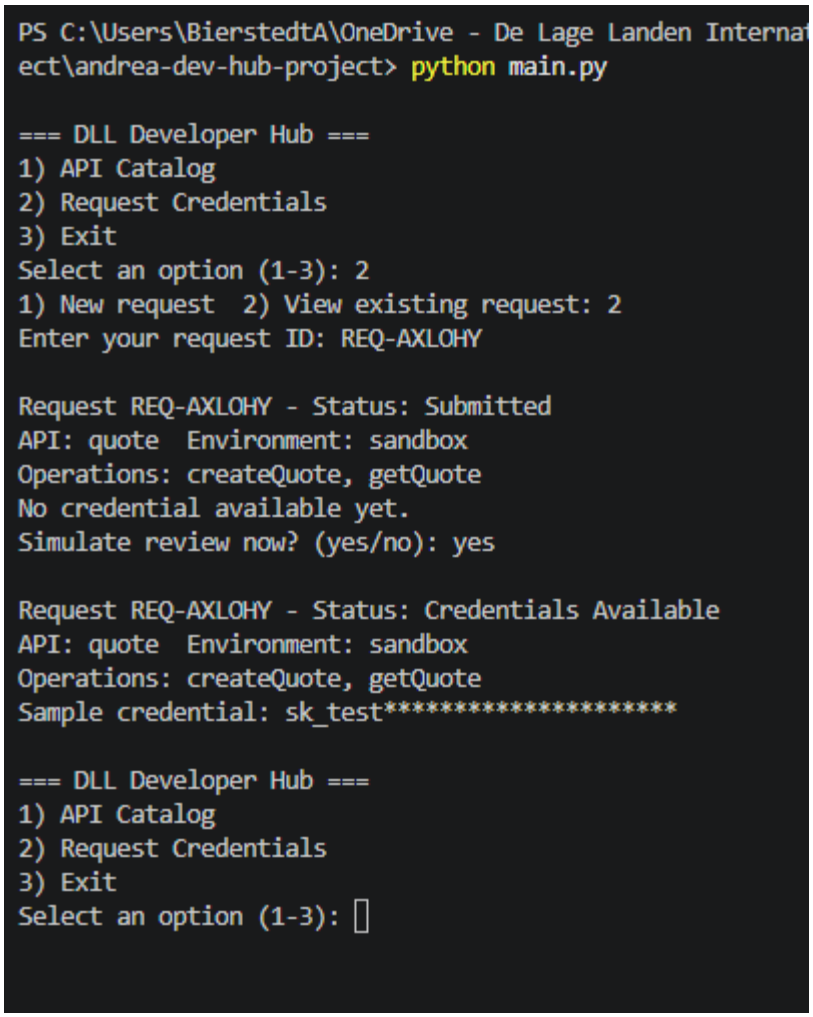
Request submitted! Your request ID is REQ-AXLOHY.
Use 'View existing request' with this ID to check its status.

Request REQ-AXLOHY - Status: Submitted
API: quote  Environment: sandbox
Operations: createQuote, getQuote
No credential available yet.

=== DLL Developer Hub ===
1) API Catalog
2) Request Credentials
3) Exit
Select an option (1-3): 3
```

3.4 Checking a request and the simulated review

Entering the request ID shows the status. Running the simulated review moves it to Credentials Available and shows a masked sample key.



3.5 Needs Information

If the user hasn't reviewed the API guide (or has no technical contact), the review moves the request to Needs Information with a reviewer note instead of issuing a key.

```
=== DLL Developer Hub ===
1) API Catalog
2) Request Credentials
3) Exit
Select an option (1-3): 2
1) New request  2) View existing request: 2
Enter your request ID: REQ-5F9ZGS

Request REQ-5F9ZGS - Status: Submitted
API: contract  Environment: production
Operations: getPaymentSchedule
No credential available yet.
Simulate review now? (yes/no): yes

Request REQ-5F9ZGS - Status: Needs Information
API: contract  Environment: production
Operations: getPaymentSchedule
Reviewer note: Please review the API guide and provide a technical contact.
No credential available yet.

=== DLL Developer Hub ===
1) API Catalog
2) Request Credentials
3) Exit
Select an option (1-3):
```

4. Issues encountered and how I solved them

- Setting up Git and GitHub. Git was not installed on my computer at first, and my work computer is signed in to an enterprise GitHub account that cannot push to my personal repo. *Solved:* installed Git, signed VS Code into my personal GitHub account, and do all commits/pushes from VS Code.
- Running the app from the wrong folder. Running `python main.py` from the parent folder failed because the data files are loaded with relative paths (`data/...`). *Solved for now:* open the repo folder itself in VS Code. *See unsolved issues below for a permanent fix.*
- Simulated approval was too simple. My first version approved every request immediately, which skipped the statuses in my spec. *Solved:* requests are now saved as "Submitted" and a separate review step moves them to "Credentials Available" or "Needs Information" based on the readiness answers.
- Bad input. Typing an API id that didn't exist, a made-up environment, or an email without "@" was accepted. *Solved:* added checks for all three.
- Commits. In report B all of my changes went into one commit because I didn't stage files individually. *Solved:* undid the commit, staged and committed each file separately, and force-pushed.
- Converting my spec to PDF. The automated Word-to-PDF conversion failed on my OneDrive file, so the first spec PDF lost its formatting. *Solved:* the revised spec is written in Markdown and exported to PDF.

5. Unsolved issues and how I plan to handle them

- Test data ends up in `data/requests.json`. Every test run adds requests to this file, and I have to remember to reset it to `[]` before committing. *Plan:* add `data/requests.json` to `.gitignore`. `load_data()` already starts with an empty list when the file doesn't exist.
- The app only works when started from the repo folder. *Plan:* build file paths from the location of `main.py` (`os.path.dirname(__file__)`) instead of the current folder.
- The command-line functions print directly. Streamlit can't reuse functions that `print()` and `input()`. *Plan:* in Version 2, separate the "logic" functions (search, validate, save, review) from the "display" functions, so the Streamlit app can import the logic from `main.py`.
- Some spec details aren't in Version 1 yet: "related APIs" on the product page, starting a credential request directly from a product page, and a "reveal key" option. *Plan:* move these to Version 2 (see the revised spec).

6. Where I need help

- Streamlit navigation. Streamlit's built-in multi-page apps use a `pages/` folder, but our class rule is that all `.py` files must be at the project root. Is it OK to build navigation in one `app.py` using `st.session_state` (and buttons/tabs) instead of a `pages/` folder?
- Simulated review. Right now the user triggers the review from "View existing request". Is that a reasonable way to simulate an approval for this class, or would you prefer something like an "admin" menu option?
- `requests.json` in the repo. Should I commit an empty `requests.json`, or ignore it with `.gitignore` as planned?

7. Milestones for Version 2

The milestones are in order. Each depends on the one before it unless noted.

1. V1 cleanup. Fix the file-path issue, ignore `data/requests.json`, add a "reveal key" option, and log any remaining bugs in `bugs.md`.
2. Separate logic from display. Change the core functions so they return data instead of printing, keeping the command-line version working on top of them.
3. Streamlit home page and navigation. Create `app.py` with a home page showing two large choices (API Catalog, Request Credentials), Back/Home buttons, and navigation using `st.session_state`.
4. Streamlit API Catalog. Search box, product cards, and a product page with Overview and API Specification tabs (operation picker, sample request/response).
5. Streamlit credential request and status. Request form with validation messages, a confirmation with the request ID, and a status page with a simulated review and a masked key with a reveal toggle.
6. Nice to have (time permitting). "Request credentials for this API" button on the product page, related APIs, resubmitting a "Needs Information" request, and separate sandbox/production requests.
7. Testing and final hand-in. Test all flows, update the README with Streamlit run instructions, and record a short demo video.

8. Self reflection

Are you satisfied with your progress so far?

Yes, I'm pretty satisfied with my progress so far. I think the functions I have written are good scaffolding for the rest of the application. It has been a little weird to think of an application as a group of functions because I normally think about an application as the UI that I interact with. This project has helped me see how the individual functions sit behind that interface and do most of the actual work.

Do you think you can finish the project within the next weeks?

I think I can finish the core of the project, but I'm less confident about how smoothly the UI portion will go. Writing a function with a known input and output feels pretty manageable. From my past experience with software development, the UI is where things can get complicated, especially once users start testing it. Users tend to interact with software in ways I never anticipated, and that is usually when the weird bugs start appearing.

Expectations vs. reality - what was easier, and what was more difficult than you expected?

Writing the individual functions has been easier than I expected. Breaking the application into smaller pieces makes the overall project feel much more manageable. The harder part has been some of the logic inside those functions, especially when I have to account for invalid or unexpected input. The basic function may be simple, but handling everything that could go wrong can add complexity pretty quickly.

Any light bulb moments? Did you discover something you had not expected?

My biggest light bulb moment has been realizing how much of software engineering is really data management. I receive some input, do something with it, and provide a result. A user makes a request, and the application finds, validates, stores, or transforms data to respond to that request. Thinking about the project this way has made building an application feel much less intimidating.